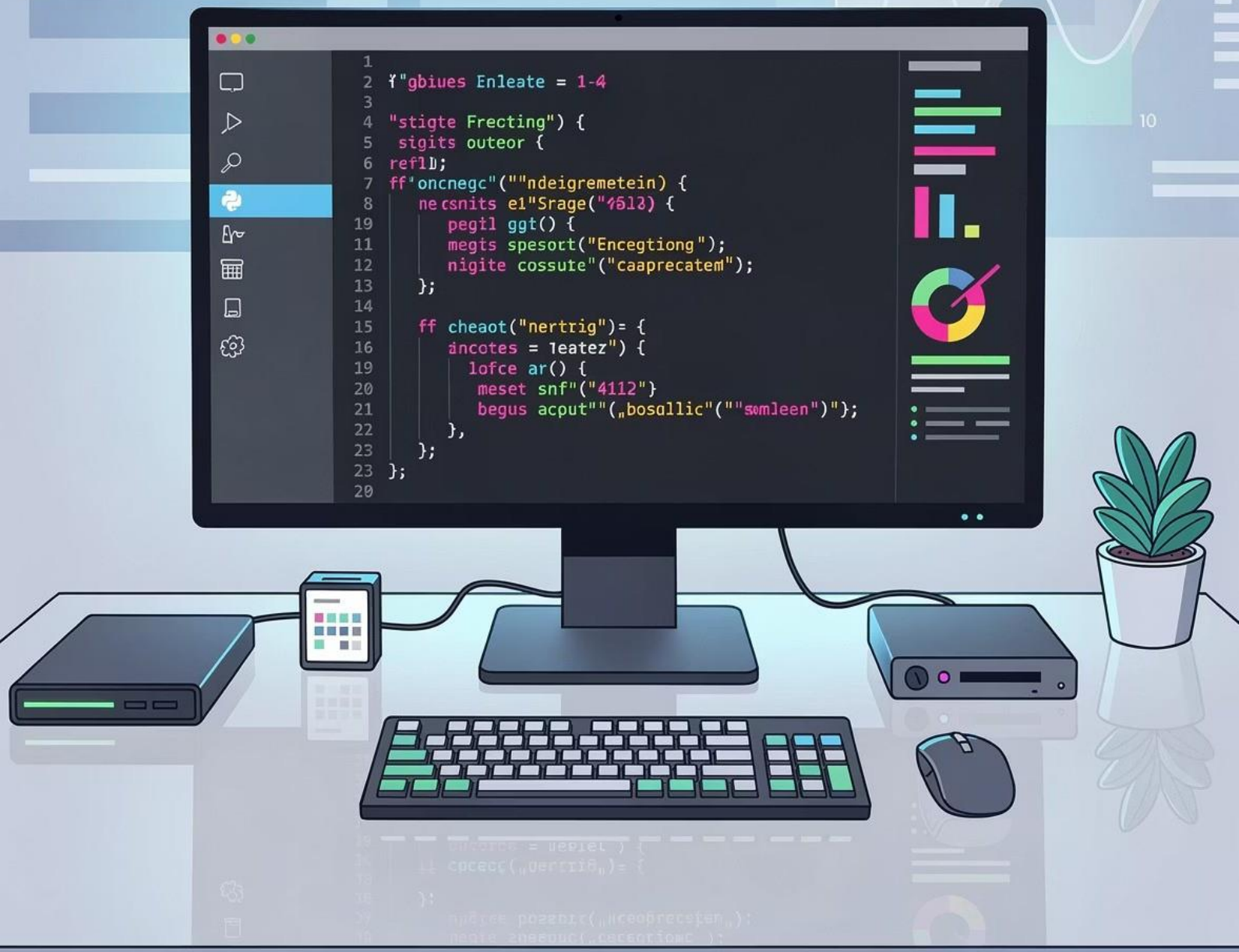
GOLDEN Q&A BANK

PYTHON | PANDAS | SQL | PYSPARK | AWS

AZURE | AIRFLOW | SCENARIOS | PROJECT



A comprehensive collection of interview questions spanning Python, Pandas, SQL, PySpark, AWS Cloud, Azure Cloud, Apache Airflow, and Scenario-Based challenges. Use this bank to prepare for technical interviews or to structure your own question-driven assessments.

PYTHON

1. What is __init__ in Python?

__init__ is the constructor method called automatically when a new object is created from a class. It initializes the object's attributes.

2. What are Decorators in Python?

Decorators wrap a function to extend its behavior without modifying it. Example: @log_time wraps a function to measure its execution time.

```
def log_time(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(time.time() - start)
        return result
    return wrapper
```

3. What is Inheritance and its types?

Inheritance lets a class reuse another class's attributes/methods. Types: **Single** (one parent), **Multiple** (many parents), **Multilevel** (chain), **Hierarchical** (many children from one parent), **Hybrid** (combination).

4. What is a Lambda function?

An anonymous, one-line function. Used with map(), filter(), reduce().

Example: map(lambda x: x*2, [1,2,3]) ³ [2,4,6].

5. List Comprehension vs Dictionary Comprehension

List: [x**2 for x in range(5)]. Dict: {k: v for k, v in items}. Both offer concise, readable alternatives to loops.

6. Python Libraries Used

NumPy – arrays; **Pandas** – DataFrames; **Scikit-learn** – ML; **TensorFlow** – deep learning; **Flask/Django** – APIs; **Boto3** – AWS SDK; **PySpark** – distributed processing.

7. List vs Tuple vs Set vs Dictionary

List: ordered, mutable. **Tuple**: ordered, immutable, faster. **Set**: unordered, unique values. **Dict**: key-value pairs, fast lookup. Use tuple for fixed data, set for deduplication.

8. Shallow Copy vs Deep Copy

Shallow copy copies the object but references nested objects. **Deep copy** copies everything recursively. Use `copy.deepcopy()` to avoid shared references.

9. `*args` and `**kwargs`

`*args` captures positional arguments as a tuple. `**kwargs` captures keyword arguments as a dictionary. Used for flexible function signatures.

10. Error/Exception Handling

Use `try/except/finally` blocks. Catch specific exceptions (`ValueError`, `KeyError`). Use `finally` for cleanup. Raise custom exceptions with `raise`.

11. Threading in Python

Threading allows concurrent execution of tasks using the `threading` module. Ideal for I/O-bound tasks (API calls, file reads) where waiting time can be overlapped.

12. Multi-threading vs Multi-processing

Multi-threading: shares memory, limited by GIL, good for I/O-bound. **Multi-processing**: separate memory spaces, bypasses GIL, good for CPU-bound tasks.

13. Garbage Collection in Python

Python uses **reference counting** 4 objects are deleted when their count hits zero. The `gc` module handles cyclic references using a generational garbage collector.

14. GIL (Global Interpreter Lock)

GIL is a mutex that allows only one thread to execute Python bytecode at a time. It prevents true parallelism in threads but doesn't affect multi-processing or I/O-bound concurrency.

15. Magic Methods in Python

Special Magic methods like `__str__`, `__len__`, `__eq__`, `__add__` that define how objects behave with built-in operations and functions.

16. OOP Concepts

Encapsulation: hide internal state. **Abstraction**: hide implementation. **Polymorphism**: same interface, different behavior. **Inheritance**: reuse parent class logic.

17. Batch Processing

Processing data in chunks rather than all at once. Use `chunksize` in Pandas or loop over file batches. Reduces memory usage for large datasets.

18. REST APIs 4 Build or Consume

Build with **Flask/FastAPI**. Consume with `requests` library. Use `GET/POST/PUT/DELETE` methods, handle JSON responses, and manage auth tokens/headers.

19. Security for Python Code

Use environment variables for secrets, never hardcode credentials. Use `hashlib` for hashing, HTTPS for APIs, input validation to prevent injection, and `bandit` for static analysis.

20. Method Overriding vs Overloading

Overriding: child class redefines parent method. **Overloading**: same method name with different parameters 4 Python doesn't natively support it but uses `*args` or `@singledispatch`.

21. Constructors in Python

`__init__` is the instance constructor. `__new__` creates the object before `__init__` initializes it. Default constructor takes only `self`; parameterized takes additional args.

22. Handle Errors in REST APIs

Check HTTP status codes (4xx = client error, 5xx = server error). Use `try/except` with `requests.exceptions`. Implement retries with exponential backoff.

23. Handle Missing Data in Pipeline

Detect with `df.isnull()`. Fill with defaults (`fillna`), forward fill, or drop rows. For critical fields, raise errors. Log missing records for audit.

24. JDBC Connections in Python

Use `jaydebeapi` or `pyodbc` to connect via JDBC. Pass connection string, driver, credentials. Fetch results into Pandas DataFrames. Common for connecting to Oracle, SQL Server.

25. Ingest Data from External API

Use `requests.get(url, headers=headers)`, parse JSON response, convert to DataFrame, validate schema, and write to target (S3, DB). Handle pagination and rate limits.

PANDAS

1. DataFrame vs Series

Series: 1D labeled array. **DataFrame**: 2D table with rows and columns. DataFrame is a collection of Series sharing the same index.

2. Create a DataFrame from Python Data

`pd.DataFrame({'col1': [1,2], 'col2': [3,4]})` 4 from dict. Also from list of dicts, NumPy arrays, or CSV/JSON files.

3. Read Different File Formats

`pd.read_csv()`, `pd.read_json()`, `pd.read_parquet()`, `pd.read_excel()`. Parquet is most efficient for large datasets due to columnar storage.

4. Handle Null/Missing Values

`df.fillna(0)` fills with a value. `df.dropna()` removes rows with nulls. `df.fillna(method='ffill')` forward fills. Use `df.isnull().sum()` to detect nulls.

5. Remove Duplicate Rows

`df.drop_duplicates()` removes all duplicate rows. Use `subset=['col']` to check specific columns. `keep='first'` or `keep=False` to control which to drop.

6. Join/Merge Multiple DataFrames

Use `pd.merge(df1, df2, on='key', how='inner')`. Chain multiple merges. For same-structure DataFrames, use `pd.concat([df1, df2])`.

7. Merge vs Join vs Concat

Merge: SQL-style join on columns. **Join**: index-based merge shortcut. **Concat**: stacks DataFrames vertically or horizontally without key alignment.

8. Rename a Column

`df.rename(columns={'old': 'new'}, inplace=True)`. Or reassign: `df.columns = ['a','b','c']`. Use `inplace=True` to modify in place.

9. Group Data with groupby()

`df.groupby('dept')['salary'].mean()`. Supports aggregations: `sum`, `count`, `max`, `agg({'col': 'sum'})`. Returns a GroupBy object until aggregated.

10. apply() on Columns

`df['col'].apply(lambda x: x*2)` applies a function element-wise. `df.apply(func, axis=1)` applies row-wise. Slower than vectorized ops but flexible.

11. Create New Columns

`df['new_col'] = df['a'] + df['b']`. Use `assign()` for chaining: `df.assign(total=df['a']+df['b'])`. Apply functions with `apply()` for complex logic.

12. Pivot a DataFrame

`pivot()` reshapes without aggregation (requires unique index). `pivot_table()` allows aggregation for duplicate values. Use `aggfunc='sum'` for rollups.

13.Handle Large Datasets

Use `chunksize` in `read_csv()`, **Dask** for parallel processing on multi-core, or **Modin** as a drop-in Pandas replacement for speed. Move to PySpark for truly large data.

14. Date-Time Data in Pandas

Parse with `pd.to_datetime()`. Extract components: `df['date'].dt.year`, `.dt.month`. Resample time-series: `df.resample('M').sum()`.

15. Combine Multiple CSVs

`pd.concat([pd.read_csv(f) for f in glob.glob('*.csv')])`. Use `ignore_index=True` to reset index. Process in chunks if files are large.

16. Join Two Large Datasets Efficiently

Set join keys as index before merging. Reduce memory by downcasting dtypes. Use `merge` with `on` key. For very large data, use chunked merges or switch to PySpark/Dask.

17. Explode Nested JSON with Pandas

```
import pandas as pd, json
df = pd.json_normalize(data, record_path=['items'],
   meta=['id','name'])
```

18.Handle Errors in Pandas

Use `try/except` around I/O operations. Use `errors='coerce'` in `pd.to_numeric()` to turn invalid values to NaN. Validate schema before processing.

19. Read Excel and Convert to CSV

```
df = pd.read_excel('file.xlsx')
df.to_csv('output.csv', index=False)
```

20. Create DataFrame with 2 Columns

```
df = pd.DataFrame({
   'file_name': ['a.txt','b.csv'],
   'size': [1024, 2048]
})
```

21. Join DataFrames and Fill NaN in 'age'

```
merged = pd.merge(df1, df2,
   on='user_id', how='left')
merged['age'].fillna(30, inplace=True)
```

22. Write DataFrame to CSV

Pandas: `df.to_csv('output.csv', index=False)`. PySpark: `df.write.csv('s3://bucket/path', header=True)`. For S3, use Boto3 or Spark's S3 connector.

23. Data Structures in Pandas

Series: 1D labeled array. **DataFrame**: 2D table. **Index**: row/column labels. **MultiIndex**: hierarchical indexing for complex groupings. Panel was removed in newer versions.

24. map() vs apply() in Pandas

`map()` works element-wise on a Series only, ideal for simple value substitutions. `apply()` works on both Series and DataFrames, supports complex functions, and can operate row-wise (`axis=1`) or column-wise.

SQL

1. Relational vs Non-Relational Databases

Relational: structured, SQL, schema-enforced (MySQL, PostgreSQL, Redshift). **Non-Relational**: flexible schema, NoSQL (MongoDB, DynamoDB, Cassandra). Use NoSQL for unstructured or high-scale data.

2. OLAP vs OLTP

OLTP: transactional, row-based, real-time updates (MySQL). **OLAP**: analytical, columnar, aggregations on large datasets (Redshift). OLAP reads wide; OLTP writes frequently.

3. Normalization and Its Types

1NF: atomic values. **2NF**: no partial dependencies. **3NF**: no transitive dependencies. Advantage: reduces redundancy. Disadvantage: more joins, slower queries.

4. De-normalization

Combining tables to reduce joins. Used in OLAP/data warehouses for query performance. Trade-off: data redundancy increases but read speed improves significantly.

5. Views in SQL

A virtual table defined by a SELECT query. Benefits: simplify complex queries, add security layer, hide implementation. Data is not stored 4 computed at query time.

6. Materialized View

Stores query results physically. Faster reads than regular views. Must be refreshed manually or on schedule. Used for pre-aggregated reporting tables in data warehouses.

7. Window Functions

`ROW_NUMBER()`: unique rank. `RANK()`: gaps on ties. `DENSE_RANK()`: no gaps. `LAG/LEAD`: access previous/next row. `SUM OVER(PARTITION BY)`: running totals. All use the `OVER()` clause.

8. Types of JOINs

INNER: matching rows only. **LEFT**: all left + matching right. **RIGHT**: all right + matching left. **FULL OUTER**: all rows from both. **CROSS**: cartesian product. INNER and LEFT are most common.

9. ROW_NUMBER vs RANK vs DENSE_RANK

For values 10, 10, 20: **ROW_NUMBER**: 1,2,3. **RANK**: 1,1,3 (skips 2). **DENSE_RANK**: 1,1,2 (no gap). Use DENSE_RANK when gaps in ranking are undesirable.

10. Left Join vs Left Outer Join

They are identical. `LEFT JOIN` is shorthand for `LEFT OUTER JOIN`. Both return all rows from the left table plus matching rows from the right.

11. Self Join vs Left Join

Self Join: joins a table to itself (e.g., employee-manager hierarchy). **Left Join**: joins two different tables. Self join uses table aliases to distinguish the two references.

12. Cross Join (Cartesian Product)

Returns every combination of rows from both tables. N × M rows. Used for generating combinations. No ON clause needed. Dangerous on large tables 4 can produce millions of rows.

13. Find Users Above Average Transaction

```
SELECT user_id, SUM(amount) AS total
FROM transactions
GROUP BY user_id
HAVING SUM(amount) > (
  SELECT AVG(total) FROM (
    SELECT user_id, SUM(amount) AS total
    FROM transactions GROUP BY user_id
  ) t
);
```

14. What are the types of constraints in SQL?

SQL constraints are rules applied on columns to maintain data integrity. Main constraints are NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, CHECK and DEFAULT.

15. Left Join vs Left Anti Join

Left Join: returns all left rows + matched right. **Left Anti Join**: returns only left rows with NO match in right (i.e., `WHERE right.id IS NULL`). Used to find orphan records.

16. CTE 4 Common Table Expression

A named temporary result set defined with `WITH`. Improves readability, allows recursion, and can be referenced multiple times. CTEs are not stored 4 recomputed each reference unless materialized.

17. CTE vs Subquery

CTE: readable, reusable within query, supports recursion. **Subquery**: inline, can't be reused. CTEs are generally preferred for complex queries; subqueries for simple one-off filters.

18. CTE vs View

CTE: temporary, exists only for the query duration. **View**: persisted in database, reusable across sessions. Use CTEs for one-time complex logic; Views for frequently reused query definitions.

19. Indexing in SQL

An index speeds up data retrieval. Types: **Clustered** (physical sort), **Non-Clustered** (pointer-based), **Composite** (multi-column), **Unique**, **Full-Text**, **Bitmap**. Avoid over-indexing 4 slows writes.

20. Stored Procedure

A precompiled SQL block stored in the DB. Advantages: reusability, reduced network traffic, security (execute without seeing code), better performance through execution plan caching.

21. Clustered vs Non-Clustered Index

Clustered: physically reorders table data, one per table, faster for range queries. **Non-Clustered**: separate structure with pointers, multiple per table, faster for specific lookups.

22. Alias in SQL

Temporary name for a table or column: `SELECT salary AS pay FROM emp e`. Aliases improve readability and are required for self joins and subquery references.

23. COUNT(*) vs COUNT(id)

`COUNT(*)` counts all rows including NULLs. `COUNT(id)` counts only non-NULL values in the `id` column. Use `COUNT(*)` for total row count.

24. Primary Key vs Composite Key

Primary Key: single column uniquely identifying rows. **Composite Key**: combination of two or more columns forming a unique identifier. Use composite keys when no single column is unique.

25. Surrogate Key vs Primary Key

Surrogate keys are system-generated (auto-increment IDs) with no business meaning. Used when natural/composite primary keys are complex, long, or subject to change. Improves join performance.

26. WHERE vs HAVING

WHERE: filters rows before aggregation. **HAVING**: filters groups after `GROUP BY`. You cannot use aggregate functions in WHERE → use HAVING for those conditions.

27. GROUP BY vs ORDER BY vs HAVING

GROUP BY: groups rows for aggregation. **ORDER BY**: sorts result set. **HAVING**: filters groups post-aggregation. They can be combined: GROUP BY → HAVING → ORDER BY.

28. SQL Order of Execution

FROM ³ JOIN ³ WHERE ³ GROUP BY ³ HAVING ³ SELECT ³ DISTINCT ³ ORDER BY ³ LIMIT. Understanding this order helps debug queries and write correct conditions.

29. UNION vs UNION ALL

UNION: combines results and removes duplicates (slower). **UNION ALL**: combines all results including duplicates (faster). Use UNION ALL unless deduplication is needed.

30. CASE Without ELSE

Yes 4 if no `ELSE` is specified, the result defaults to `NULL` when no WHEN condition matches. Always add ELSE for safety in production queries.

31. IN vs BETWEEN

IN: matches specific discrete values (`WHERE id IN (1,2,3)`). **BETWEEN**: range of values inclusive on both ends (`WHERE age BETWEEN 18 AND 30`). BETWEEN is cleaner for ranges.

32. Subquery vs Correlated Subquery

Subquery: independent, runs once. **Correlated Subquery**: references outer query, runs per row (slower). Use CTEs or joins instead of correlated subqueries for performance.

34. DELETE vs DROP vs TRUNCATE

DELETE: DML, removes rows with condition, can rollback. **TRUNCATE**: DDL, removes all rows fast, no rollback in most DBs. **DROP**: DDL, removes the entire table structure and data.

35. SQL Optimization Techniques

Use indexes, avoid SELECT *, use EXISTS instead of IN for large sets, avoid functions on indexed columns in WHERE, use partitioning, analyze query execution plans, avoid nested loops.

36. DELETE Without Condition

Yes, `DELETE FROM table`; removes all rows but logs each deletion 4 slower and more resource-intensive than TRUNCATE. TRUNCATE is preferred for clearing tables due to speed and efficiency.

37. Window Function vs Aggregate Function

Aggregate collapses rows into one result (GROUP BY). **Window Function** computes across a window of rows without collapsing them 4 each row retains its identity. Use window functions for rankings, running totals, and lead/lag comparisons.

PYSPARK

1. Spark Architecture

Driver: orchestrates execution, creates DAG. **Cluster Manager**: allocates resources (YARN, Mesos, K8s). **Executors**: run tasks on worker nodes. **DAG Scheduler**: breaks DAG into stages and tasks.

2. Components of Spark

Spark Core, Spark SQL, Spark Streaming, MLlib (machine learning), GraphX (graph processing). Spark SQL and DataFrames are most commonly used in data engineering.

3. SparkSession vs SparkContext

SparkContext: original entry point for RDD-based Spark. **SparkSession**: unified entry point (Spark 2.0+) that includes SparkContext, SQLContext, and HiveContext. Always use SparkSession in modern code.

4. RDD 4 Resilient Distributed Dataset

RDD is Spark's core abstraction: immutable, distributed collection of objects. Features: fault-tolerant (via lineage), lazy evaluation, partitioned across nodes. Lower-level than DataFrames.

5. RDD vs DataFrame vs Dataset

RDD: untyped, no optimization. **DataFrame**: tabular, Catalyst-optimized, Python/Scala/R. **Dataset**: typed DataFrame, Scala/Java only. DataFrames are preferred for data engineering.

6. DataFrame in Spark

A distributed table with named columns and schema. Backed by Catalyst optimizer for query optimization. Created from files (CSV, Parquet, JSON), databases, or RDDs via `spark.read.`

7. Lazy Evaluation

Spark doesn't execute transformations immediately 4 it builds an execution plan (DAG). Execution only happens at an **action** (`count`, `collect`, `write`). This enables optimization before execution.

8. Catalyst Optimizer

Spark's built-in query optimizer. It analyzes the logical plan, applies rules (predicate pushdown, column pruning), and generates an optimized physical execution plan using cost-based optimization.

9. Predicate Pushdown

Pushing filter conditions as close to the data source as possible (e.g., into Parquet files or databases), reducing the amount of data read into memory before processing.

10. Column Pruning

Catalyst drops unnecessary columns early in the execution plan so only required columns are read from storage. Especially effective with columnar formats like Parquet.

11. Fault Tolerance in Spark

Spark uses **lineage** 4 it records the sequence of transformations to recreate lost partitions. If an executor fails, Spark recomputes only the lost partition from its lineage graph, not the entire job.

12. DAG in Spark

A Directed Acyclic Graph represents the execution plan 4 nodes are RDDs/DataFrames, edges are transformations. Spark creates the DAG at action trigger, then optimizes and submits it to the scheduler.

13. Stages and Tasks

Stage: a set of transformations that don't require shuffling. **Task**: a unit of work sent to one executor for one partition. Wide transformations (shuffle) create stage boundaries.

14. Narrow vs Wide Transformations

Narrow: each input partition maps to one output partition, no shuffle (`map`, `filter`, `select`). **Wide**: requires data shuffling across partitions (`groupBy`, `join`, `distinct`). Wide transformations are expensive.

15. Types of Joins in Spark

Inner, Left, Right, Full Outer, Cross, Left Anti, Left Semi. Spark chooses join strategy (broadcast, sort-merge, shuffle-hash) based on data size and configuration.

16. Broadcast Join

Small table is broadcast (copied) to all executors, avoiding shuffle. Use when one table is small (< 10MB default). Enable: `spark.conf.set("spark.sql.autoBroadcastJoinThreshold", 10MB)` or use `broadcast(df)` hint.

17. Sort-Merge Join vs Shuffle-□ash Join

Sort-Merge: both datasets sorted and merged 4 scalable for large datasets. **Shuffle-Hash**: builds hash map in memory 4 faster but requires one side to fit in memory. Spark auto-selects.

18. Cache vs Persist

cache(): stores in memory (MEMORY_AND_DISK by default). **persist()**: allows specifying storage level (MEMORY_ONLY, DISK_ONLY, OFF_HEAP, etc.). Use for reused DataFrames to avoid recomputation.

19. Storage Levels in Spark

MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, MEMORY_ONLY_SER (serialized), MEMORY_AND_DISK_SER, OFF_HEAP. Choose based on data size and available memory.

20. Repartition vs Coalesce

Repartition: full shuffle, increases or decreases partitions. **Coalesce**: no full shuffle, only decreases partitions by merging. Use Coalesce to reduce partitions efficiently; Repartition when increasing.

21. Repartition 4 Wide or Narrow?

Repartition: wide transformation (involves full shuffle). **Coalesce**: narrow transformation (merges existing partitions without full shuffle).

22. Decrease Partitions: Repartition vs Coalesce

Both decrease partition count. **Repartition** shuffles all data equally. **Coalesce** merges partitions on the same node with no shuffle 4 much cheaper. Prefer Coalesce for reduction.

23. AQE 4 Adaptive Query Execution

AQE dynamically optimizes query execution at runtime based on actual statistics (partition sizes, skew). Enable: `spark.sql.adaptive.enabled=true`. Handles skew, dynamic partition coalescing automatically.

24. Optimization Techniques in Spark

Use Parquet/columnar formats, broadcast joins, partition pruning, predicate pushdown, AQE, cache reused DataFrames, avoid UDFs (use built-in functions), tune executor memory/cores, address skew with salting.

25. Apache Spark vs Hadoop MapReduce

Spark: in-memory, 100x faster, supports streaming, SQL, ML. MapReduce: disk-based, write intermediate to HDFS, simpler but slow. Spark is the modern replacement for most MapReduce use cases.

26. Data Skewness

Skew occurs when some partitions have far more data than others (e.g., one key = 90% of rows). Causes: hot keys, non-uniform distribution. Effects: slow tasks, OOM. Handle with salting, AQE, or repartitioning.

27. Salting in Spark

Add a random salt prefix to skewed keys to distribute data evenly. Join on salted key after broadcasting small table with same salted copies. Remove salt post-join.

```
df = df.withColumn("salt_key",
   concat(col("key"),
   lit("_"),
   (rand()*10).cast("int")))
```

28. Does More Executors Solve Skew?

No 4 skew means one partition is overloaded. More executors don't help if one task handles 90% of the data. You must redistribute data with salting, AQE, or repartitioning on a better key.

29. Dynamic Number of Executors

Enable Dynamic Resource Allocation: `spark.dynamicAllocation.enabled=true.` Spark scales executors based on pending tasks. Set min/max: `spark.dynamicAllocation.minExecutors` / `maxExecutors.`

30. Partitions for 5 GB Data

Default partition size is 128MB. For 5GB: ~40 partitions. Workers/nodes depend on cluster config. Rule of thumb: 2-4 partitions per CPU core. More partitions = better parallelism but more overhead.

31. Default Partitions in Spark

`spark.default.parallelism`: for RDD operations (default 200 or 2× cores).
`spark.sql.shuffle.partitions`: for DataFrame shuffle operations (default 200). Tune based on data size.

32. Partition Strategy

Partition on high-cardinality columns used in filters/joins. Avoid skewed partition keys. For time-series data, partition by date. Target 128MB–256MB per partition. Use `repartition(n, col)`.

33. Schema Evolution in Spark

Use `mergeSchema=true` option when reading Delta/Parquet. Handle new columns with `PERMISSIVE` mode. Use Glue Dynamic Frames for flexible schema. Add default values for missing columns.

34. Read Different File Formats in PySpark

`spark.read.csv()`, `.json()`, `.parquet()`, `.orc()`, `.avro()`, `.jdbc()`, `.format("delta").load()`. Specify `schema` and `options` as needed.

35. df.read Options

Key options: `header=True`, `inferSchema=True`, `delimiter=','`, `mode='PERMISSIVE'` (for corrupt records), `mergeSchema=True`, `multiLine=True` for JSON.

36. Write Data to Different Targets

S3: `df.write.parquet("s3://bucket/path")`. Redshift: use JDBC or `spark-redshift` connector with temp S3. Delta: `df.write.format("delta").save(path)`. Use `mode("overwrite"/"append")`.

37. Incremental Loads in Spark

Filter by max watermark: `df.filter(col("updated_at") > last_run_date)`. Use Delta Lake MERGE for upserts. Use Glue Job Bookmarks or Airflow execution dates as watermarks.

38. Handle Skewed Data in Spark

Techniques: **Salting** hot keys, **AQE** auto-skew handling, **repartition** on a better column, **broadcast join** for small tables, **filter separately** then union skewed and non-skewed partitions.

39. Monitor Spark Jobs

Spark UI (port 4040): view DAGs, stages, tasks, storage. CloudWatch for Glue/EMR. Ganglia/Prometheus for cluster metrics. Check executor logs for OOM errors. Use `df.explain()` for plan analysis.

40. Checkpointing in Spark

Saves RDD/DataFrame to stable storage (HDFS/S3) to break long lineage chains. Prevents recomputation from scratch on failure. Essential for long iterative ML algorithms and streaming.

41. Small File Problem

Many tiny files hurt performance (NameNode pressure, slow reads). Solutions: use `coalesce()` before writing, compact with Delta `OPTIMIZE`, configure Glue to produce fewer output files.

42. Handle OOM Errors in Spark

Increase executor memory (`spark.executor.memory`). Reduce data loaded (filter, select). Use `persist(DISK_ONLY)`. Increase partitions to reduce per-partition size. Avoid `collect()` on large datasets.

43. Reduce Runtime for Long Jobs

Cache reused DataFrames. Enable AQE. Use broadcast joins. Increase parallelism. Use columnar formats (Parquet). Avoid UDFs. Partition smartly. Check for skew and shuffle bottlenecks.

44. What Happens After spark-submit?

Driver starts ³ requests resources from Cluster Manager ³ executors launched on worker nodes ³ DAG created ³ stages/tasks distributed to executors ³ results returned to driver or written to storage.

45. Handle Data Inside PySpark

Read with `spark.read`, transform with DataFrame API (`select`, `filter`, `join`, `groupBy`), validate schema, handle nulls, and write with `df.write`. Cache intermediate results if reused.

46. Handle Bottlenecks in Spark

Identify via Spark UI (long stages, skewed tasks). Common fixes: increase partitions, add executor memory, fix data skew, use broadcast joins, cache hot DataFrames, tune shuffle partitions.

47. Monitor and Optimize Nodes

Use Spark UI to identify idle executors or overloaded nodes. Balance workload via repartition. Use dynamic allocation. Monitor GC time 4 high GC means memory pressure. Right-size executor cores and memory.

48. df.explain()

Shows the physical and logical execution plan. Use `df.explain(True)` for full plan including parsed, analyzed, optimized logical and physical. Helps identify missing predicate pushdown or inefficient joins.

49. GroupByKey vs ReduceByKey

GroupByKey: shuffles all data to reducers, then groups 4 memory-heavy. **ReduceByKey**: pre-aggregates on each partition before shuffle 4 far more efficient. Always prefer ReduceByKey for aggregations.

50. Create Explicit Partition in PySpark

```
df.write.partitionBy("year","month")\
   .parquet("s3://bucket/path")
```

At read time, Spark uses partition pruning to read only matching directories.

51. Add New Column with withColumn

```
df = df.withColumn("new_col",
   col("price") * col("qty"))
df = df.withColumn("flag",
   lit(1))
```

52. Define Schema Explicitly

```
from pyspark.sql.types import *
schema = StructType([
   StructField("id", IntegerType(), True),
   StructField("name", StringType(), True)
])
df = spark.read.schema(schema).csv("path")
```

53. Window Functions in PySpark

```
from pyspark.sql.window import Window
from pyspark.sql.functions import rank
w = Window.partitionBy("dept")\
   .orderBy("salary")
df.withColumn("rank", rank().over(w))
```

54. SQL in PySpark

```
df.createOrReplaceTempView("sales")
result = spark.sql("""
   SELECT region, SUM(amount)
   FROM sales GROUP BY region
""")
```

55. Write a UDF in PySpark

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

@udf(StringType())
def clean(s): return s.strip().upper()

df = df.withColumn("col",clean("col"))
```

Prefer built-in functions over UDFs 4 UDFs can't be optimized by Catalyst.

56. Accumulators in Spark

Shared variables updated only by executors (additive only). Driver reads the final value. Used for counting errors, events, or skipped records across tasks without returning full data to driver.

57. Yesterday's vs Today's Data

Filter by date column: `filter(col("date") == current_date())` or `date_sub(current_date(), 1)`. In Airflow, use `execution_date` and `prev_execution_date` as parameters.

58. Broadcast Variables

Read-only shared variables cached on each executor to avoid repeated data transfer. Use for lookup tables: `bc = sc.broadcast(lookup_dict)`. Accessed in tasks via `bc.value`.

59. Is distinct() Wide or Narrow?

Wide transformation 4 it requires shuffling data across partitions to identify and remove duplicates globally. This creates a stage boundary in the DAG.

60. Unpersist Cached Data

Yes: `df.unpersist()` removes the DataFrame from cache. Call this after the DataFrame is no longer needed to free executor memory for other tasks.

61. Delete/Drop Physical Partition

```
spark.sql("""
   ALTER TABLE db.table
   DROP IF EXISTS
   PARTITION (year='2024',month='01')
""")
```

This removes both the metadata and the underlying data files.

62. Can you Delete Data from Partitions 16320 in a 5GB read? Explain.

You cannot selectively delete internal Spark partitions by index 4 partitions are internal implementation details. Instead, filter your data and overwrite: `df.filter(condition).write.mode("overwrite")`.

63. Serialization vs Deserialization in PySpark

Serialization: converting objects to byte stream for network transfer or storage.
Deserialization: reconstructing objects. Spark uses Java or Kryo serializer 4 Kryo is faster and more compact.

64. df.groupBy()

Groups rows by one or more columns for aggregation. Returns a `GroupedData` object. Must be followed by an aggregation (`count()`, `sum()`, `agg()`). Example: `df.groupBy("dept").agg(sum("salary"))`.

AWS

1. Amazon S3

Simple Storage Service 4 object storage for any data type. Key features: virtually unlimited storage, 99.999999999% durability, event triggers, lifecycle policies, versioning, and cross-region replication.

2. S3 Storage Classes

Standard: frequent access. **Standard-IA**: infrequent access, cheaper. **One Zone-IA**: one AZ, lower cost. **Glacier**: archival, minutes to hours retrieval. **Glacier Deep Archive**: cheapest, 12-hour retrieval.

3. S3 Standard vs Standard-IA vs One Zone-IA

Standard: highest availability, 3+ AZs. **Standard-IA**: 3+ AZs, lower cost, retrieval fee. **One Zone-IA**: 1 AZ only, cheapest but lower resilience. Use One Zone-IA for re-creatable data.

4. S3 Security

IAM Roles: service-level access. **Bucket Policies**: resource-based JSON policies. **ACLs**: legacy object-level access. **KMS**: encryption key management. Use bucket policies + IAM roles as primary controls.

5. S3 Encryption Types

SSE-S3: AWS-managed keys. **SSE-KMS**: customer-controlled keys in KMS. **SSE-C**: customer-provided keys. **Client-side**: encrypt before uploading. SSE-KMS is most common for data engineering.

6. Versioning and Encryption in S3

Enable versioning in bucket settings 4 all versions of objects are kept. Combine with lifecycle rules to expire old versions. Enable default encryption so all objects are automatically encrypted on upload.

7. S3 Limitations

Max object size: 5TB (multipart upload required for > 5GB). Max PUT request: 5GB. Request rate: 3,500 PUT/5,500 GET per prefix per second. S3 offers strong read-after-write consistency since 2021.

8. Bucket Policies

JSON-based resource policies attached to S3 buckets. Control who can access what actions on which resources. Example: allow cross-account access or restrict to specific VPC endpoints only.

9. Data After Landing in S3

Trigger Lambda or Glue job via S3 event notification. Validate/transform data, convert to Parquet, move to processed prefix. Update Glue Catalog via Crawler. Load to Redshift or query with Athena.

10. Optimize S3 Storage Costs

Use Intelligent-Tiering for unknown access patterns. Apply lifecycle policies to transition to Glacier after N days. Delete expired versions. Compress files. Use Parquet instead of CSV to reduce size.

11. Trigger Transformation on File Arrival

Use S3 Event Notification → trigger Lambda (for small files) or SNS/SQS → Glue job (for large files). EventBridge can also route S3 events to Step Functions for complex orchestration.

12. Copy Files Between Buckets via CLI

```
aws s3 cp s3://source-bucket/ \
  s3://target-bucket/ \
  --recursive
```

Use `--exclude` / `--include` patterns to filter files. Add `--sse aws:kms` to encrypt in transit.

13. Automate CSV/Parquet Ingestion from SharePoint to S3

Use Microsoft Graph API to authenticate and download files. Schedule with Lambda or Glue Python shell. Upload to S3 using Boto3 `s3.put_object()`. Trigger downstream pipeline on success.

14.Handle Duplicate Files in S3 with Boto3

Generate MD5 hash of incoming file. Compare against stored hashes in DynamoDB or S3 metadata. Skip or archive duplicates. Use `s3.head_object()` to check if file already exists.

15. Read Today's File and Archive Rest

Use Boto3 to list objects filtered by date prefix. Download today's file. Move older files: `s3.copy_object()` to archive bucket, then `s3.delete_object()` from source.

16. S3 in Data Pipeline

S3 is the landing zone, staging, and data lake storage layer. Raw files land ³ Lambda triggers Glue ETL ³ Parquet stored in processed prefix ³ Athena/Redshift Spectrum queries directly from S3.

17. S3 vs Hadoop File System (HDFS)

S3: object storage, decoupled from compute, pay per use, durable. HDFS: block storage, co-located with compute, requires running cluster. S3 is preferred for cloud-native pipelines; HDFS for on-prem Hadoop.

18. Access Data Across Two AWS Accounts

Use **Cross-Account IAM Roles**: account B assumes a role from account A. Set bucket policy in account A to allow account B's role. Use `sts.assume_role()` in code to get temporary credentials.

19. Why Use S3 Before Glue?

S3 acts as a decoupled staging layer 4 decouples ingestion from processing. Glue can read directly from any S3 path without locking the source. Also provides durability, easy reprocessing, and audit trails.

20. Extract-Transform-Move to S3 Pipeline

Extract from source ³ Glue or Lambda transforms ³ write Parquet to target S3. File size consideration: aim for 128MB–256MB Parquet files to avoid small file problem in downstream queries.

21. Check Data 7 Days Back with Lifecycle Policy

If files were transitioned to Glacier, restore them using `restore-object` API (takes hours). If S3 Versioning was enabled, access previous versions. Always test lifecycle policies in non-prod first.

22. AWS Glue

Serverless ETL service. Features: Glue Catalog, Crawlers, ETL jobs (Spark/Python shell), Workflows, Studio (visual ETL), Job Bookmarks for incremental loads, support for G.1X/G.2X workers.

23. Glue Crawler

Automatically scans data stores (S3, RDS, JDBC) and infers schema. Populates the Glue Data Catalog with table definitions. Can be scheduled or triggered by events. Detects schema changes.

24. Glue Data Catalog

Central metadata repository 4 stores table definitions, schema, partition info, and data location. Works with Athena, Redshift Spectrum, and EMR. Acts like a Hive Metastore for AWS services.

25. Glue Database

A logical container in the Glue Catalog that groups related tables. Each Glue Database corresponds to a schema or domain (e.g., `raw_db`, `curated_db`). Queried using Athena or Spark SQL.

26. Schema Evolution in Glue

Use Glue Dynamic Frames with `choose_one_schema` or `resolve_choice()` to handle conflicting types. Enable `updateBehavior="UPDATE_IN_DATABASE"` in Crawler. Use Parquet with `mergeSchema=True`.

27. Glue Dynamic Frames vs Spark DataFrames

Dynamic Frame: schema-flexible, handles mismatched types, supports Glue-specific transforms (`resolveChoice`, `relationalize`). **DataFrame**: schema-strict, Catalyst-optimized, faster for clean data.

28. Data Lakehouse with S3 + Glue + Athena

Store raw/processed data in S3 (Parquet/Delta). Glue Crawler updates Catalog. Athena queries directly via SQL. Glue ETL handles transformation. Add Redshift Spectrum for BI tool integration.

29. Parallel Processing in Glue

Glue jobs run distributed Spark under the hood. Use concurrent runs for independent datasets. Set `--enable-job-insights` and tune DPUs. Each Glue job spawns multiple Spark executors across DPU workers.

30. DPU in Glue / Worker Types

DPU: Data Processing Unit 4 4 vCPU, 16GB RAM. **G.1X**: 1 DPU per worker, standard workloads. **G.2X**: 2 DPU per worker, memory-intensive. **G.025X**: Python shell jobs, lightweight tasks.

31. Where Glue Jobs Run

Glue jobs run on AWS-managed infrastructure (not your VPC by default). You can configure Glue to run inside your VPC by providing VPC, subnet, and security group settings 4 required for accessing RDS or private endpoints.

32. Glue Job Bookmarks

Track which data has been processed so only new/changed data is reprocessed on the next run. Enable with `--job-bookmark-option job-bookmark-enable`. Works with S3, JDBC sources.

33. Types of Glue Jobs

Spark ETL: distributed PySpark jobs. **Python Shell**: lightweight Python scripts (G.025X). **Streaming**: continuous Spark Streaming jobs. **Ray**: distributed Python for ML workloads (newer).

34. Glue Studio

Visual drag-and-drop ETL authoring tool in Glue. Build pipelines without writing code 4 generates Glue script underneath. Good for simple transformations; prefer code for complex logic and version control.

35. Glue vs EMR

Glue: serverless, no cluster management, higher cost per DPU, easier setup. **EMR**: managed Hadoop/Spark clusters, more control, cheaper for long-running workloads, requires cluster management expertise.

36. Glue vs Lambda

Lambda: <15 min, small files, event-driven, stateless. **Glue**: long-running ETL, large datasets, distributed Spark. Use Lambda for lightweight triggers and Glue for heavy transformations.

37. Micro-ETL in Glue

Small, focused Glue Python shell jobs that handle specific, lightweight tasks (e.g., copy a single file, update metadata). Cost-efficient using G.025X workers. Good for modular pipeline design.

38. Deploy Glue Code via CI/CD

Store Glue scripts in GitHub/CodeCommit. Use CodePipeline or GitHub Actions to validate, test, and deploy scripts to S3. Update Glue job to point to new script version. Use CloudFormation/Terraform for infra.

39. External Packages in Glue

Upload libraries as `.whl` or `.zip` to S3. Reference in job parameters: `--additional-python-modules` for PyPI packages or `--extra-py-files` for custom packages from S3.

40. Glue Workflows

Orchestrate multiple Glue jobs and crawlers with triggers (time, event, or dependency). Create sequential or parallel pipelines. Use triggers to chain: Crawler ³ Glue ETL Job ³ next Job.

41. Glue Job Duration and Optimizations

Target: <30 min for most jobs. Optimizations: increase DPUs, use Parquet, enable job bookmarks, partition data, use pushdown predicates, avoid small files, cache reused DynamicFrames.

42. Worker Types Used in Glue

G.1X for standard workloads. G.2X for memory-intensive joins/aggregations. G.025X for Python shell jobs. Choose based on data volume 4 monitor SparkUI to right-size.

43. Glue Context

`GlueContext` extends SparkContext 4 it's the entry point for Glue-specific APIs (reading Dynamic Frames, writing to Glue Catalog, using Glue transforms). Created alongside SparkSession in Glue scripts.

44. Why Crawler Runs Every Ingestion?

To detect new partitions or schema changes in S3. Without re-crawling, new partitions aren't registered in the Catalog and Athena can't query them. Use `MSCK REPAIR TABLE` as a lighter alternative.

45. How Glue Detects New Files in S3

Glue Crawler scans S3 paths on schedule or trigger. Alternatively, S3 Event Notifications ³ EventBridge ³ Lambda or Step Functions ³ trigger Glue job directly when new files arrive.

46. Missing Columns in Dynamic Frames

Dynamic Frames handle missing columns natively 4 missing columns are null. Use `resolveChoice()` to cast ambiguous types. Use `applyMapping()` to explicitly define expected schema and fill defaults.

47. Glue Pipeline Best Practices

Partition input data, use job bookmarks, enable Spark UI, write outputs as Parquet, handle errors with retries, use Glue Catalog for schema management, separate raw/processed S3 prefixes, monitor job metrics.

48. 1 G.1X vs 2 G.2X Workers

2 G.2X workers (4 DPU total) is better than **1 G.1X** (1 DPU) for memory-intensive work. G.2X gives more memory per worker and better parallelism. However, 2 G.2X costs more 4 profile before scaling.

49. Time-Based Triggers 4 Glue vs Lambda

Glue: built-in schedule triggers using cron expressions in Workflows. **Lambda**: use EventBridge (CloudWatch Events) with cron expression to invoke Lambda on schedule. Both support cron syntax.

50. Minimum Nodes in Glue

Minimum is **2 workers** (1 driver + 1 executor) for Spark ETL jobs. For G.025X Python Shell jobs, 1 node is sufficient. You cannot go below 2 workers for distributed Spark Glue jobs.

51. AWS Lambda

Serverless compute service 4 run code without provisioning servers. Event-driven, auto-scales, pay per invocation. Supports Python, Node.js, Java, etc. Ideal for lightweight event processing.

52. Lambda Limitations

Max timeout: 15 minutes. Deployment package: 50MB (zipped), 250MB (unzipped). Concurrent executions: 1,000 (default, can increase). Memory: 128MB to 10GB. Ephemeral storage: up to 10GB in /tmp.

53. Max Execution Time and Package Size

Timeout: **15 minutes** max. Package size: **50MB** compressed ZIP (250MB uncompressed). For larger dependencies, use Lambda Layers or container images (up to 10GB).

54. Is 15-Minute Timeout Standard?

Yes 4 15 minutes is the hard maximum and cannot be extended. It's configurable from 1 second to 15 minutes. For longer processing, use Step Functions, Glue, or Fargate instead.

55. Max Concurrent Connections in Lambda

Default: 1,000 concurrent executions per account per region. Can be increased by submitting a service quota request. Reserve concurrency per function to isolate critical functions from quota exhaustion.

56. Load Custom Layers to Lambda

Via UI: Layers ³ Create Layer ³ upload ZIP. Without UI: `aws lambda publish-layer-version --zip-file`. Attach to function via `--layers` ARN. Max 5 layers per function.

57. Pandas Layers in Lambda

Pandas isn't included in Lambda runtime by default. Add it as a Lambda Layer (pre-built AWS-managed layer or custom ZIP). Required for DataFrame operations in Lambda without bundling in deployment package.

58. Lambda Triggers

S3 events, API Gateway, EventBridge (scheduled/event), SQS, SNS, DynamoDB Streams, Kinesis, Cognito, CloudFront (Lambda@Edge). Each trigger type has different payload formats and concurrency behavior.

59. Create Event Trigger in Lambda

In console: Lambda ³ Add Trigger ³ select S3/EventBridge/SQS. Via CLI: `aws lambda create-event-source-mapping` for SQS/Kinesis. For S3: configure bucket event notification to point to Lambda ARN.

60. Load 1000 Files from S3 to Redshift via Lambda

Use SQS queue 4 each S3 event sends a message. Lambda processes messages in batches. Each Lambda invocation loads one batch using COPY command or JDBC. Use DLQ for failed messages.

61. Lambda Calling Another Lambda

Yes 4 use `boto3.client('lambda').invoke()`. Use `InvocationType='Event'` for async or `'RequestResponse'` for sync. For complex chaining, prefer Step Functions over Lambda-to-Lambda calls.

62. Methods to Deploy Lambda

Console ZIP upload, AWS CLI, SAM (Serverless Application Model), CloudFormation, CDK, Terraform, container image via ECR. CI/CD: CodePipeline + CodeDeploy or GitHub Actions.

63. Lambda vs Glue for 1MB File

Use **Lambda** – 1MB is tiny. Lambda starts in milliseconds, no cluster spin-up time. Glue takes 1–2 min to start. Lambda is cheaper, faster, and more appropriate for small event-driven processing.

64. Glue Direct vs Lambda Chunking

For large files: **Glue** is better – distributed processing handles volume efficiently. For medium files needing chunking: **Lambda** with SQS chunking can be more cost-efficient if data fits within timeout limits.

65. Build a Data Lake in AWS

Landing (S3 raw) → Glue ETL transforms → Processed S3 (Parquet, partitioned) → Glue Catalog metadata → Athena for ad-hoc queries → Redshift Spectrum for BI → IAM + KMS for governance.

66. Is Athena a Database?

No – Athena is a serverless interactive query service. It queries data in S3 using standard SQL. Creates **external tables** (data stays in S3, only metadata in Glue Catalog). No storage layer.

67. Advantages of Athena

Serverless – no infrastructure. Pay per query (per TB scanned). Works directly on S3. Uses Glue Catalog. Supports Parquet, ORC, CSV, JSON. Good for ad-hoc analysis without loading data into a database.

68. Athena vs Redshift

Athena: serverless, pay-per-query, S3-native, good for ad-hoc. **Redshift**: fully managed data warehouse, persistent cluster (or serverless), better for complex analytics and BI tools. Use both together.

69. Redshift Spectrum vs Athena

Both query S3 using Glue Catalog. **Spectrum**: runs from inside Redshift cluster, can join S3 data with Redshift tables. **Athena**: fully independent, no Redshift cluster needed. Athena is simpler for pure S3 queries.

70. Optimize Athena Performance

Use Parquet/ORC columnar formats. Partition data by date/region. Use compression (Snappy, GZIP). Limit SELECT columns. Use partition pruning in WHERE. Avoid `SELECT *`. Use larger files (128MB+).

71. Create External Table in Athena from Parquet

```
CREATE EXTERNAL TABLE db.sales (
 id INT, amount DOUBLE
)
STORED AS PARQUET
LOCATION 's3://bucket/sales/'
TBLPROPERTIES  ('parquet.compression'='SNAPPY');
```

72. Amazon Redshift Architecture

Leader Node: receives queries, creates execution plan, distributes to compute nodes. **Compute Nodes**: execute queries in parallel on slices. **Slices**: partitions within each compute node. Uses columnar storage + MPP.

73. MPP in Redshift

Massively Parallel Processing 4 query work is distributed across all compute nodes simultaneously. Each node processes its slice of data in parallel, dramatically improving performance for large analytical queries.

74. Columnar Storage in Redshift

Data stored column-by-column instead of row-by-row. Benefits: only reads needed columns (column pruning), better compression (same-type values compress better), faster aggregation for analytical queries.

75. Single Load into Redshift 4 COPY Command

```
COPY schema.table
FROM 's3://bucket/path/'
IAM_ROLE 'arn:aws:iam::ID:role/role'
FORMAT AS PARQUET;
```

COPY is the most efficient way to bulk-load data 4 uses parallel loading from S3.

76. Incremental Data in Redshift

Use watermark (max `updated_at`) to extract only new records. Stage new data in S3 ³ COPY to staging table ³ MERGE/UPDATE/INSERT into main table. Use Delta tables or transaction IDs for CDC.

77. Redshift Spectrum

Extends Redshift to query data directly in S3 without loading it. Uses external tables in Glue Catalog. Benefits: query both Redshift and S3 data in one SQL, no ETL needed for cold data, cost-effective.

78. Optimize Redshift Query Performance

Define Sort Keys (range scans). Choose Distribution Keys (avoid data movement). Use compression encodings. Run VACUUM and ANALYZE regularly. Use materialized views. Enable result caching. Check WLM queues.

79. Data Lake vs Data Warehouse vs Data Lakehouse

Data Lake: raw, unstructured, cheap (S3). **Data Warehouse**: structured, curated, fast queries (Redshift). **Lakehouse**: combines both 4 open formats (Delta/Iceberg) on cloud storage with warehouse-like performance.

80. Redshift Serverless

No cluster to manage 4 Redshift auto-scales capacity based on workload. Pay per RPU (Redshift Processing Unit) per second. Ideal for variable or unpredictable query patterns. Automatically pauses when idle.

81. Redshift vs Querying S3 with Athena

Redshift: persistent cluster, better for complex joins, BI tools, consistent performance. Athena: serverless, ad-hoc, pay-per-query. Use Athena for exploration; Redshift for production dashboards and SLA-bound queries.

82. Types of Views in Redshift

Standard View: virtual, recomputed at query time. **Materialized View**: stored result, faster. **Late Binding View**: references external (Spectrum) tables without strict schema binding — useful for schema evolution.

83. Distribution Styles in Redshift

EVEN: round-robin, balanced. **KEY**: same key values on same node. **ALL**: entire table on every node (small tables). **AUTO**: Redshift decides. Syntax: `DISTSTYLE KEY DISTKEY(col)`.

84. Dist Key vs Sort Key

Distribution Key: determines which node stores each row (reduces data movement in joins). **Sort Key**: determines physical sort order on disk (speeds up range filters). Use both strategically for performance.

85. EC2 Instance Types

General: **t3/m5**. Compute: **c5/c6g**. Memory: **r5/x1**. Storage: **i3/d2**. GPU: **p3/g4**. Micro: **t2**. Choose based on workload — memory-optimized for Spark, compute-optimized for ML inference.

86. Connect to EC2 from VS Code via SSH

Install Remote-SSH extension. Configure `~/.ssh/config` with EC2 public IP and key file path. In VS Code: Remote Explorer → connect to host. Ensure EC2 security group allows port 22.

87. Amazon CloudWatch

Monitoring and observability service. Collects metrics, logs, events. Set alarms on thresholds. Create dashboards. Monitor Glue jobs, Lambda, EC2, Redshift. Use Log Groups for centralized log storage.

88. Debug Logs and Metrics in CloudWatch

Use CloudWatch Logs Insights for querying log data. Filter by error level. Set metric filters to extract patterns from logs. Use CloudWatch Alarms to notify on anomalies. Correlate with X-Ray for tracing.

89. When a Pipeline Fails

Check CloudWatch logs for error details. Identify root cause (data issue, infra, code). Fix and rerun. If partial completion, use idempotent design 4 reprocess only failed partitions. Notify stakeholders via SNS.

90. Common Pipeline Failure Causes

Bad source data (schema change, null values), OOM errors, timeout, network issues, missing files, permission errors (IAM), dependency failures, wrong configuration, environment-specific bugs.

91. Send Notifications on Pipeline Failure

CloudWatch Alarm on Glue/Lambda error metric ³ SNS topic ³ email/Slack via Lambda. Or Airflow email/Slack operator on failure callbacks. Use `on_failure_callback` in Airflow tasks.

92. IAM Users, Groups, Roles, Policies

User: individual with long-term credentials. **Group**: collection of users. **Role**: temporary credentials, assumed by services/accounts. **Policy**: JSON document defining permissions. Use roles for services, not users.

93. AWS KMS

Key Management Service 4 create, manage, and rotate encryption keys. Used to encrypt S3, RDS, Redshift, Glue data. Customer Managed Keys (CMK) give full control. Enable via resource encryption settings.

94. AWS DMS and CDC

Database Migration Service migrates data between sources and targets. CDC (Change Data Capture) uses database transaction logs to capture inserts/updates/deletes in near real-time. Supports ongoing replication.

95. File Formats Worked With

CSV, JSON, Parquet, Avro, ORC, Delta, XML. Parquet is preferred for analytical workloads. Avro for streaming (Kafka). Delta for ACID transactional data lakes. ORC for Hive/EMR workloads.

96. AWS CloudFormation

Infrastructure as Code (IaC) service. Define AWS resources in YAML/JSON templates. CloudFormation provisions and manages the stack. Enables repeatable, version-controlled infrastructure deployments across environments.

97. AWS Step Functions

Orchestration service using state machines. Define workflows with JSON (Amazon States Language). Chain AWS services (Lambda, Glue, ECS). Supports branching, retries, parallel execution. Better than Lambda chaining for complex flows.

98. AWS Secrets Manager

Stores and rotates credentials (DB passwords, API keys) securely. Access via API or SDK. Supports automatic rotation for RDS/Redshift. Integrates with Lambda, Glue. Eliminates hardcoded secrets in code.

99. Parquet vs CSV Advantages

Columnar (reads only needed columns), compressed (Snappy/GZIP), schema embedded, 2–10× smaller, 10–100× faster for analytical queries. Supports predicate pushdown and partition pruning. Ideal for Athena and Redshift Spectrum.

100. SCD Types

Type 1: overwrite old value. **Type 2**: add new row with effective dates, keep history. **Type 3**: add previous value column. **Type 4**: history in separate table. Type 2 is most common in data warehouses.

101. Incremental Loads Without DMS

Use watermarks (`updated_at` column). Extract records where `updated_at > last_run_time`. Store last run time in DynamoDB or S3 metadata file. Update watermark after successful load.

102. Fault Tolerance and High Availability in AWS

Multi-AZ deployments, S3 multi-region replication, Redshift Multi-AZ, Lambda auto-scaling, SQS for decoupling, CloudWatch alarms, retry logic, dead letter queues, cross-region failover with Route 53.

103. Design a Scalable Data Pipeline on AWS

Source ³ S3 landing ³ EventBridge triggers ³ Glue ETL ³ S3 processed (Parquet, partitioned) ³ Glue Catalog ³ Athena/Redshift ³ CloudWatch monitoring ³ SNS alerts. Use Step Functions for orchestration.

104. Deploy Code from Git to AWS

Use CodePipeline: Git push ³ CodeBuild (build/test) ³ CodeDeploy/CloudFormation (deploy). For Glue: copy scripts to S3. For Lambda: package and deploy via SAM/CDK. Use branches for environment separation.

105. Trigger Glue Job When Data Arrives in S3

S3 Event Notification ³ EventBridge rule ³ trigger Glue job directly or via Lambda. Or use CloudTrail + EventBridge pattern matching on `PutObject` events. Glue Workflows can use S3-based triggers too.

106. Move Data from S3 to Redshift

COPY command (most efficient 4 parallel load). **Glue ETL** with Redshift connector. **Lambda + psycopg2** for small files. **Redshift Data API** for serverless. Always use IAM role, not credentials, in COPY.

107. AWS Kinesis / Streaming Data

Kinesis Data Streams: real-time data ingestion (like Kafka). Kinesis Firehose: managed delivery to S3/Redshift. Kinesis Analytics: SQL on streaming data. Used for IoT, clickstream, fraud detection use cases.

108. Very High Data Volume via JDBC

Partition JDBC reads: set `partitionColumn`, `lowerBound`, `upperBound`, `numPartitions` in Spark JDBC options. This enables parallel reads from database into multiple Spark partitions simultaneously.

109. Internal DB to S3 via JDBC in Python Operator

Use `cx_Oracle` or `pyodbc` to connect, execute query, fetch data to DataFrame, write to S3 using Boto3 or `df.to_parquet(s3_path)` via `s3fs`. Schedule with Airflow PythonOperator.

AZURE

1. Why Use Azure Data Factory?

ADF is a cloud ETL/ELT orchestration service. Used to ingest data from diverse sources (SQL, APIs, files), schedule pipelines, transform data using Data Flows or Databricks, and monitor pipeline health.

2. Data Sources Connected Using ADF

Azure SQL, On-prem SQL Server (via Self-hosted IR), SharePoint, REST APIs, Azure Blob/ADLS, Salesforce, Oracle, SAP. ADF supports 90+ connectors via Linked Services.

3. Incremental Loading in ADF

Use watermark pattern: store max `LastModifiedDate` in a lookup table. Each run reads records newer than the watermark. Update watermark on success. Alternatively, use ADF's built-in incremental copy with Change Tracking.

4. ADF Activities Used

Copy Activity, Lookup Activity, ForEach, If Condition, Execute Pipeline, Web Activity (REST calls), Databricks Notebook Activity, Stored Procedure Activity, Get Metadata, Delete Activity.

5. Handle Pipeline Failures in ADF

Set retry count/interval on activities. Use failure dependency arrows to trigger cleanup/alert pipelines. Monitor via ADF Monitor tab. Send alerts via Logic Apps or Azure Monitor. Log errors to a database or ADLS.

6. ADF Triggering Databricks Notebooks

Use Databricks Notebook Activity in ADF. Configure Linked Service with Databricks workspace URL and PAT token (or managed identity). Pass parameters via Base Parameters. Monitor notebook run status in ADF.

7. Credentials Secured in ADF

Store secrets in Azure Key Vault. Reference in Linked Services using Key Vault Linked Service 4 ADF retrieves secrets at runtime. Never hardcode credentials. Use Managed Identity for Azure service authentication.

8. Why Use Azure Databricks?

Managed Apache Spark environment on Azure. Used for large-scale data transformations, ML model training, Delta Lake operations, and collaborative notebook-based development. Integrates natively with ADLS and ADF.

9. Transformations in Databricks

Deduplication, null handling, type casting, joins, aggregations, window functions, SCD Type 2 logic, feature engineering, JSON parsing, and data quality checks 4 all using PySpark or Spark SQL.

10. Databricks Read/Write from ADLS Gen2

Mount ADLS using `dbutils.fs.mount()` or use abfss:// path directly. Authenticate with Service Principal (OAuth) or Managed Identity. Read: `spark.read.parquet("abfss://...")`. Write similarly.

11. Databricks for ML Workflows

Use MLflow for experiment tracking, model registry, and deployment. Databricks Feature Store for feature tables. AutoML for baseline models. Delta tables for training datasets versioning. MLflow integrates natively.

12. Pass Parameters from ADF to Databricks

In ADF Databricks Notebook Activity ³ Base Parameters: add key-value pairs. In notebook, access via `dbutils.widgets.get("param_name")`. Supports dynamic expressions using ADF system variables.

13. Manage Databricks Clusters

Use **Job Clusters** (auto-created per job, cheaper) vs **All-Purpose Clusters** (interactive, always on). Enable auto-scaling and auto-termination to control costs. Right-size node types (memory vs compute optimized).

14. Log Errors in Databricks

Use Python `logging` module or `print()` 4 all output goes to notebook stdout. Use `try/except` and log errors to Delta tables or ADLS for audit. Integrate with Azure Monitor for centralized alerting.

15. Why ADF + Databricks Together?

ADF handles orchestration, scheduling, and data movement. Databricks handles heavy transformations. Each does what it's best at 4 ADF is the workflow engine; Databricks is the processing engine. Separation of concerns.

16. Ensure Data Quality

Validate schema at ingestion. Check null counts, row counts, and value ranges. Use Delta constraints or custom PySpark checks. Log quality metrics. Fail pipeline on critical violations. Use Great Expectations or Databricks DQ.

17. End-to-End Churn Prediction Pipeline

Ingest raw events via ADF ³ Bronze ADLS. Databricks cleans/transforms ³ Silver. Feature engineering ³ Gold feature tables. ML model trained in Databricks MLflow. Predictions written back to Delta. ADF schedules daily.

18. Incremental Loads in ADF

Watermark-based: Lookup activity reads last watermark ³ Copy Activity with filter ³ stored procedure updates watermark. Or use ADF's native incremental copy with Change Tracking on Azure SQL source.

19. Build Reusable ADF Pipelines

Use parameters and variables. Create template/master pipelines that call sub-pipelines with Execute Pipeline activity. Use ForEach for iteration. Store config in a metadata table read via Lookup Activity.

20. Ingest Large-Scale Clickstream Data in Databricks

Use Event Hubs/Kafka ³ Databricks Structured Streaming. Process micro-batches. Write to Delta Bronze table with `appendOnly=True`. Use Auto Loader for efficient incremental file ingestion from ADLS.

21. Why Delta Lake in Churn Projects?

ACID transactions prevent corrupt data. Time travel for reproducible ML training. MERGE for upserts (update customer features). Schema evolution for new behavioral signals. Better performance than plain Parquet.

22. Optimize Slow Databricks Jobs

Check Spark UI for bottlenecks. Enable AQE. Use Delta table `OPTIMIZE` + `ZORDER`. Cache reused DataFrames. Broadcast small lookup tables. Fix data skew. Use Photon engine. Right-size cluster.

23. Data Quality Checks

Row count validation (source vs target). Null percentage on critical columns. Value range checks. Duplicate detection. Schema validation. Log results to a DQ metrics table. Alert on failures before proceeding downstream.

24. Secure PII Data

Mask PII at ingestion (hash, tokenize, redact). Encrypt ADLS with CMK. Use Unity Catalog for column-level security. Restrict access via IAM/RBAC. Audit with Databricks audit logs. Apply Delta column masking policies.

25. Design Churn Feature Tables

Use Databricks Feature Store. Tables include: user activity (sessions, clicks), product engagement (views, purchases), recency/frequency/monetary (RFM) metrics. Partition by date. Register features with metadata for reuse.

26. Production Failures and Monitoring

Set Azure Monitor alerts on ADF pipeline failures. Use Databricks job alerts. Implement idempotent pipelines for safe reruns. Log all job runs with status. Use on-call rotation and runbook for common failure types.

27. Optimize Cost in Azure Databricks

Use Spot/Preemptible instances. Enable auto-termination. Use Job Clusters instead of All-Purpose. Right-size nodes. Use Delta OPTIMIZE to reduce storage. Enable Photon for faster compute per DBU. Monitor DBU usage.

28. On-Prem SQL Server to Databricks via ADF

Install Self-Hosted Integration Runtime on on-prem server. Create Linked Service for SQL Server. ADF Copy Activity ³ source: SQL query, sink: ADLS Gen2 (Parquet). Databricks reads from ADLS. Schedule in ADF trigger.

29. Load PySpark DataFrame into Delta Table

```
df.write.format("delta")\
    .mode("overwrite")\
    .saveAsTable("catalog.schema.table")
# Or append:
df.write.format("delta")\
    .mode("append").save("/path/delta")
```

30. Backend Files in Delta Table

Delta creates: **Parquet data files** (actual data), **_delta_log/** directory (transaction log JSON files). Log files record all operations (add, remove, metadata). This log enables ACID transactions and time travel.

31.Hive Metastore vs Unity Catalog

Hive Metastore: workspace-scoped, no cross-workspace sharing, limited governance. **Unity Catalog**: account-level, multi-workspace, fine-grained access (column, row), lineage tracking, centralized governance.

32. Managed vs External Tables

Managed: Databricks controls data and metadata 4 drop table deletes data. **External**: only metadata managed, data stays in ADLS 4 drop table leaves data intact. Use external for shared/persistent datasets.

33. Parameterize a Databricks Notebook

```
dbutils.widgets.text("start_date","2024-01-01")
start = dbutils.widgets.get("start_date")
```

Pass values from ADF Databricks Activity ³ Base Parameters. Use widgets for interactive parameter input.

34. Can Notebooks Have Input and Output Parameters?

Yes 4 **input** via dbutils.widgets. **Output** via dbutils.notebook.exit("value"). ADF reads the exit value as output parameter and can use it in downstream activities.

35. Output Parameters in Real-World Pipelines

A Databricks notebook returns a count of processed records via dbutils.notebook.exit(str(count)). ADF reads this and decides via If Condition: if count > 0, proceed; else, skip downstream steps.

36. Do We Need All Three Medallion Layers?

Not always. For simple pipelines, Bronze³Gold may suffice. Silver adds value when complex cleansing is needed before aggregation. Always have Bronze (raw) as a safety net. Gold serves the business use case directly.

37. What is Databricks?

A unified data analytics platform built on Apache Spark. Provides collaborative notebooks, managed Spark clusters, Delta Lake, MLflow, and Unity Catalog. Available on Azure, AWS, and GCP as a managed service.

38. Architecture of Databricks

Control Plane: Databricks-managed (web app, cluster manager, job scheduler, notebooks). **Data Plane**: runs in your cloud account (Spark clusters, storage). Data never leaves your subscription.

39. Which Plane Has Compute and Data?

Data Plane → Spark clusters (compute) and ADLS/S3/GCS storage (data) both run within your cloud subscription. This ensures data sovereignty. Control Plane only manages metadata and orchestration.

40. Databricks Secrets

Encrypted key-value store within Databricks. Backed by Azure Key Vault. Access via `dbutils.secrets.get(scope, key)`. Secrets are never displayed in notebook output. Create scopes backed by Key Vault for enterprise use.

41. Incremental Load Using ADF

Lookup Activity reads last watermark → Copy Activity filters source (`WHERE updated_at > @watermark`) → sink to ADLS → Stored Procedure Activity updates watermark table with new max date on success.

42. Move Data from Bronze to Silver

Databricks reads Bronze Delta table → applies transformations (dedup, null handling, type casting, schema validation) → writes to Silver Delta table using `MERGE` or `overwrite`. Log row counts for audit.

43. Tightly Coupled Architecture

Components are interdependent → changes in one break others. Example: ETL code hardcoded to specific schema. Hard to scale, test, or modify independently. Opposite of microservices or modular design.

44. Fully Decoupled Architecture

Components communicate via interfaces/events → no direct dependencies. Example: producers write to S3/Event Hub; consumers read independently. Enables independent scaling, deployment, and failure isolation.

45. Migrate On-Prem SQL to Databricks via ADF

Self-Hosted IR on on-prem → ADF Linked Service → Copy Activity (SQL source → ADLS Parquet sink) → Databricks converts to Delta in Silver layer → validate row counts match source.

46. Load PySpark DataFrame into Delta Table

See question 29 – same answer. Use `df.write.format("delta").mode("overwrite"/"append").saveAsTable()` or `.save(path)`. Use `MERGE` for upserts via `DeltaTable.forPath()`.

47. Backend Files When Writing to Delta

See question 30 – Parquet data files + `_delta_log/` JSON transaction log files. The log contains commit history, schema, and file operations. Run `VACUUM` to clean old files and `OPTIMIZE` to compact.

48. Hive Metastore vs Unity Catalog

See question 31 – Unity Catalog is the modern replacement offering account-level governance, lineage, column-level security, and multi-workspace sharing. Hive Metastore is workspace-scoped and lacks fine-grained controls.

49. Parameterize a Databricks Notebook

See question 33 – use `dbutils.widgets.` Best practice: define all widgets at the top of the notebook. Use text, dropdown, combobox, or multiselect widget types based on input requirements.

50. Notebooks with Input and Output Parameters

See question 34 – input via `dbutils.widgets.get()`, output via `dbutils.notebook.exit()`. In ADF, the exit value is captured in the notebook activity output and accessible via dynamic content expressions.

51. How ADF Uses Notebook Output Parameters

The Databricks Notebook Activity in ADF exposes an `output.runOutput` field containing the notebook exit value. Reference it as `@activity('NotebookActivity').output.runOutput` in downstream ADF activities.

52. Daily File Writes in ADLS

Partition ADLS folders by date: `container/entity/year=2024/month=01/day=15/`. Use ADF dynamic expressions (`@formatDateTime(utcnow(),'yyyy/MM/dd')`) to route files to correct date partition daily.

53. Identify New Records

Use watermarks (`updated_at` or `created_at` columns). Compare source max date vs last successful run date. Alternatively, use hash-based comparison or CDC with Change Tracking / SQL Server CT feature.

54. Azure Integration Runtime (IR)

The compute infrastructure ADF uses to execute activities. **Azure IR**: cloud-to-cloud data movement. **Self-Hosted IR**: connects to on-prem/private network resources. **Azure-SSIS IR**: runs SSIS packages in cloud.

55. If Azure IR Is Not Present, Create Pipeline?

A default Azure IR is auto-created per region. You can create additional IRs for specific regions or networking requirements. Always configure Linked Services to use the appropriate IR for the data source location.

56. Copy Data Without IR?

No 4 Integration Runtime is mandatory for any ADF data movement. It's the underlying compute. Without IR, Copy Activity cannot connect to source or sink. At minimum, the default Azure IR must exist.

57. Can ADF Call APIs?

Yes 4 use the **Web Activity** to make HTTP calls (GET/POST/PUT). Used to call REST APIs, trigger Azure Functions, or invoke Logic Apps. Pass authentication headers, body, and parse JSON responses.

58. CI/CD in Azure Data Factory

Connect ADF to Azure DevOps Git. Develop in feature branches. Create Pull Requests to main. ADF export ARM template to DevOps. Pipeline deploys ARM template to Test ³ UAT ³ Production environments automatically.

59. CI/CD Conflict Scenarios in ADF

Conflicts occur when multiple developers edit the same pipeline in different branches. Also when deploying ADF changes alongside dependent Databricks notebook changes that aren't in sync across environments.

60. Why Parquet in Bronze and Delta in Silver/Gold?

Bronze: raw storage with minimal transformation 4 Parquet is lightweight and open. Silver/Gold: need ACID transactions, time travel, MERGE capability 4 Delta Lake provides these. Delta is built on Parquet with the transaction log on top.

61. Incremental Loads from Azure SQL

Enable Change Tracking on Azure SQL. ADF Lookup reads last sync version. Copy Activity uses `@{activity('Lookup').output.firstRow.SYS_CHANGE_VERSION}` to extract only changed rows. Update version on success.

62. How Key Vault Helps

Centralized secret management 4 connection strings, passwords, SAS tokens. ADF, Databricks, and Azure services retrieve secrets at runtime via managed identity. Eliminates credentials in code or config files. Supports automatic rotation.

63. Design Fact and Dimension Tables for AR

Fact: `fact_invoice` (invoice_id, customer_id, date_id, amount, status). Dims: `dim_customer`, `dim_date`, `dim_aging_bucket`. Use surrogate keys, SCD Type 2 for customer history.

64. AR Aging Buckets

Aging buckets classify outstanding invoices by overdue time: **Current** (0 days), **1330 days**, **313 60 days**, **61390 days**, **90+ days**. Calculated as `DATEDIFF(today, due_date)` categorized into bands.

65. Why Delta Lake for Healthcare Analytics?

HIPAA compliance needs audit trails 4 Delta's transaction log provides full history. Time travel for reproducibility. ACID transactions prevent partial data corruption. MERGE for patient record upserts. Schema enforcement for data integrity.

66. Manage Reruns and Failures

Design idempotent pipelines (safe to rerun). Use overwrite mode or MERGE for Delta. Log each pipeline run with status and row counts. On failure, fix root cause and rerun 4 pipeline should produce same results.

67. Role of ADF vs Databricks

ADF: orchestration, scheduling, data movement, trigger management, monitoring. **Databricks**: heavy transformation, ML, feature engineering, Delta Lake operations. ADF coordinates when and what; Databricks executes the computation.

APACHE AIRFLOW

1. What is Apache Airflow?

An open-source workflow orchestration platform. Schedules, monitors, and manages data pipelines as code using DAGs (Python). Widely used for ETL, ML pipelines, and data engineering workflows.

2. What is a DAG in Airflow?

Directed Acyclic Graph 4 a Python file defining a pipeline's tasks and their dependencies. "Directed": tasks have direction. "Acyclic": no loops. Airflow Scheduler reads DAG files and triggers them per schedule.

3. Task and Job in Airflow

Task: a single unit of work (one operator instance). **DAG Run**: one execution of the entire DAG. **Task Instance**: a specific run of a task within a DAG run. "Job" is informal 4 usually means a DAG run.

4. 20 Tasks in Parallel in Airflow

Set `concurrency` on DAG and tasks. Use `LocalExecutor` (multi-process) or `CeleryExecutor` (distributed). Set task dependencies to none between parallel tasks. Tune `max_active_tasks` per DAG.

5. Airflow Operators

PythonOperator: run Python. **BashOperator**: run shell. **GlueJobOperator**: trigger Glue. **S3Operator**: S3 operations. **SQLOperator**: run SQL. **TriggerDagRunOperator**: trigger another DAG. **HttpOperator**: REST calls. **EmailOperator**: send emails.

6. XCom in Airflow

Cross-Communication 4 pass small values between tasks. Push: `ti.xcom_push(key='result', value=data)`. Pull: `ti.xcom_pull(task_ids='task1', key='result')`. Not for large data 4 use S3/database for big payloads.

7. Write a DAG for PySpark Job

```
with DAG('spark_pipeline',
  schedule_interval='@daily') as dag:
  glue = GlueJobOperator(
  task_id='run_glue',
  job_name='my_spark_job',
  script_args={'--date':
     '{{ ds }}'})
  glue
```

8. Trigger a DAG Manually

Via Airflow UI: DAGs → click play button → Trigger DAG. Via CLI: `airflow dags trigger dag_id`. Via REST API: POST `/api/v1/dags/{dag_id}/dagRuns` with auth token and optional config JSON.

9. Ways to Trigger a DAG

Schedule (cron expression), manual trigger (UI/CLI/API), TriggerDagRunOperator (from another DAG), sensor-based (FileSensor, ExternalTaskSensor), REST API. MWAA/Cloud Composer also support EventBridge triggers.

10. Scheduler vs Executor in Airflow

Scheduler: reads DAG files, determines which tasks are ready to run, and queues them. **Executor**: runs the queued tasks. Scheduler decides what runs; Executor decides how it runs (local, Celery, Kubernetes).

11. Types of Executors in Airflow

SequentialExecutor: one task at a time (dev only). **LocalExecutor**: parallel on single machine. **CeleryExecutor**: distributed across workers. **KubernetesExecutor**: each task in a K8s pod. **CeleryKubernetesExecutor**: hybrid.

12. Store Connections in Airflow

Airflow Connections (UI/CLI): store host, login, password, extras. Encrypted in the metadata DB. Reference by `conn_id` in operators. Use Secrets Backend (HashiCorp Vault, AWS Secrets Manager) for production security.

13. Change DAG Configuration

Modify Python DAG file 4 change `schedule_interval`, `default_args`, `catchup`, `max_active_runs`, task params. Restart Scheduler to pick up changes. Use Airflow Variables for runtime config without code changes.

14. Create a DAG in Airflow

```
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG('my_dag',
   start_date=datetime(2024,1,1),
   schedule_interval='@daily',
   catchup=False) as dag:
 t1 = PythonOperator(
   task_id='task1',
   python_callable=my_func)
```

15. Task Retries in Airflow

Set in `default_args`: `retries=3, retry_delay=timedelta(minutes=5)`. Also configurable per task. Failed tasks retry automatically after delay. Use exponential backoff: `retry_exponential_backoff=True`.

16. Pipeline Running Daily, Processing Only New Data

Use Airflow's `execution_date` (or `data_interval_start`). Pass as parameter to query: `WHERE date = '{{ ds }}'`. Only processes data for the specific execution date. Set `catchup=False` for current-only runs.

17. DAG Types Used in Projects

Linear DAGs (sequential tasks), branching DAGs (BranchPythonOperator), parallel DAGs (multiple tasks no dependency), dynamic DAGs (generated programmatically from config). Most common: linear + parallel combination.

18. Downstream Task Runs Even if Upstream Fails

Use `trigger_rule='all_done'` on the downstream task. Other options: 'all_success' (default), 'one_failed', 'none_failed', 'none_skipped'. Trigger rules give fine-grained control over dependency behavior.

19.Handle Large Datasets in Airflow

Airflow is an orchestrator 4 don't process data inside Airflow tasks. Delegate to Glue, Spark, or Databricks. Use XCom only for metadata (file paths, counts). Avoid pulling large data into operator memory.

20. Airflow vs AWS Glue for Orchestration

Glue Workflows only orchestrate Glue jobs and Crawlers. Airflow orchestrates anything 4 Glue, Lambda, Redshift, Databricks, HTTP calls, bash. Airflow gives more flexibility, DAG visualization, and cross-service coordination.

21. Airflow vs Step Functions

Airflow: code-based DAGs, rich UI, complex Python logic, scheduling built-in. **Step Functions**: JSON/visual state machine, better AWS service integration, no scheduler 4 triggered by events. Airflow for scheduled pipelines; Step Functions for event-driven AWS workflows.

22. Scheduler Not Picking Up New DAGs

Check DAG parsing errors in Scheduler logs. Ensure DAG file is in the dags folder. Verify no import errors. Restart Scheduler. Check `dag_discovery_safe_mode`. Ensure `min_file_process_interval` isn't too long.

23. DAG Concurrency Issues

`max_active_runs`: limits concurrent DAG runs. `concurrency`: max tasks running at once per DAG. `depends_on_past=True`: task won't run until previous run's same task succeeded. Tune these to prevent queue overload.

24. Task Keeps Failing After Retries

Check logs for root cause (code error, connection issue, data issue). Fix the underlying problem. Mark task as success manually if safe to skip. Trigger DAG run from failed task forward. Add monitoring/alert for repeat failures.

25. Dynamic Task Mapping in Airflow

Airflow 2.3+ feature 4 generate tasks dynamically at runtime based on data. Example: `process.expand(item=get_items())` creates one task per item. Use for processing unknown number of files or partitions.

26. Monitor Jobs in Airflow

Airflow UI: DAG grid view, Gantt chart, task logs. Monitor task duration trends. Set SLA misses for alerts. Integrate with CloudWatch (MWAA), Datadog, or Prometheus. Check failed/queued task counts regularly.

27. Send Status Notifications via Email

Set `email_on_failure=True`, `email=['team@co.com']` in `default_args`. Or use EmailOperator explicitly. Configure SMTP in `airflow.cfg`. For Slack: use SlackWebhookOperator in `on_failure_callback`.

28. Define Dependencies in Airflow DAGs

Bitshift operators: `task1 >> task2` (task1 runs before task2). List: `task1 >> [task2, task3]` (parallel). `set_downstream()` / `set_upstream()` methods also work. Use `chain()` for complex sequences.

29. DAG Run vs Task Instance

DAG Run: a single execution of an entire DAG for a given `execution_date`. **Task Instance**: one specific task within that DAG run. Each DAG run contains multiple task instances 4 one per task in the DAG.

30. Monitor and Set Up Alerts in Airflow

SLA Misses: set `sla` parameter on tasks. Email alerts on failure/retry. Integrate Airflow metrics with Prometheus/Grafana. Use `on_failure_callback` to trigger custom alerting (PagerDuty, Slack, SNS).

31. DAG Fails Due to Missing Connection

Check Airflow Connections (Admin ³ Connections). Add missing connection (host, credentials). Test connection. If production, add to Secrets Backend. Rerun failed task from Airflow UI. Document all required connections in README.

32. Integrate Airflow with Glue / Deployment

Use `GlueJobOperator` from `airflow.providers.amazon.aws.operators.glue`. Provide `job_name`, `script_args`, `aws_conn_id`. Deploy Airflow on MWAA (managed), EC2, or Kubernetes (Helm chart). DAGs stored in S3 (MWAA) or local dags folder.

33. Airflow Webserver

The UI component of Airflow. Shows: DAG list, DAG runs, task instances, logs, connections, variables, XCom values, SLA misses, execution timeline, and Gantt chart. Access via browser on port 8080.

34. Incremental Loads in Airflow Pipeline

Pass `{{ ds }}` (execution date) as parameter to downstream jobs. Job processes only that day's data. Use `catchup=True` to backfill historical dates. Combine with idempotent processing to safely rerun any date.

35. Schedule a Glue Job Using Airflow

```
glue_task = GlueJobOperator(
 task_id='run_etl',
 job_name='my-glue-job',
 script_args={
 '--date': '{{ ds }}',
 '--env': 'prod'
 },
 aws_conn_id='aws_default',
 dag=dag
)
```

SCENARIO-BASED & PROJECT-BASED

1. You are ingesting daily sales data into S3. One day, the pipeline ingests duplicate records. How will you detect and handle this duplication?

Check Spark UI for skewed stages or straggler tasks. Check if source data volume increased. Look for data skew. Verify DPU count hasn't changed. Check for small file explosion. Review CloudWatch for resource constraints. Enable AQE.

2. Your Spark job is failing with an Out Of Memory (OOM) error on the executor nodes. What steps will you take to resolve it?

Increase executor memory (`spark.executor.memory`). Increase partitions to reduce per-partition size. Avoid `collect()`. Cache strategically. Use `persist(DISK_ONLY)`. Check for data skew concentrating data on one executor.

3. You are asked to implement schema evolution in your pipeline. Yesterday the source added a new column, breaking your ETL job. How will you handle it in Glue or Spark?

In Glue: use Dynamic Frames with `resolveChoice()`. In Spark: use `mergeSchema=True` for Parquet/Delta. Add schema validation at ingestion with alerts on change. Design pipelines to handle extra columns gracefully with default nulls.

4. Your Redshift query is running for hours without finishing. How do you debug and optimize it?

Run `EXPLAIN` 4 check for missing dist/sort keys or full table scans. Check WLM queue 4 query may be queued. Run `VACUUM` and `ANALYZE`. Look for table locks. Rewrite with better join order and predicate pushdown.

5. You are migrating a SQL-based on-prem data warehouse to AWS Redshift. What are the key factors you will consider to ensure performance and cost efficiency?

Assess: data volume, query patterns, dependencies. Use DMS for migration. Design dist/sort keys for top queries. Convert DDL. Test query performance. Set up COPY jobs for bulk load. Validate row counts and results before cutover.

6. An Airflow DAG failed in production due to a missing connection configuration. How will you troubleshoot and fix the issue?

Check Admin ³ Connections. Add the missing connection with correct credentials. Test it. Review why connection was missing 4 environment promotion issue? Document all required connections. Add connection existence checks to pipeline startup.

7. Your data pipeline has upstream dependencies (3 sources). One source failed, but the pipeline still ingested incomplete data. How will you implement data quality checks?

Implement pre-load validation: check row counts from each source before proceeding. Use ADF/Airflow conditions 4 only proceed if all sources pass validation. Flag incomplete loads in a run log table. Alert on failure.

8. A client requests real-time dashboards, but your system only supports batch processing. How will you redesign the pipeline to support near real-time analytics?

Introduce Kinesis Data Streams or Kafka for event streaming. Use Kinesis Firehose ³ S3 (micro-batch every 1–5 min). Databricks Structured Streaming or Glue Streaming for processing. Athena or Redshift Materialized Views for dashboards.

9. In a PySpark job, you notice data skew where one key has 90% of the records. How would you handle this issue?

Apply salting: append random suffix (0–9) to hot key. Broadcast small side table with 10 copies of each key (one per salt). Join on salted key. Remove salt after join. Alternatively, enable AQE 4 it detects skew and splits partitions automatically.

10. Your data is landing in S3 as CSV files, but analysts are complaining about query performance in Athena.How would you improve performance without changing the source system?

Convert CSV to Parquet: `df.write.parquet(s3_path)`. Partition by date or region. Update Glue Catalog. Run MSCK REPAIR TABLE. Queries now use partition pruning and column reads only 4 massive speed improvement without changing source.

11. You are handling GDPR-compliant data. How will you ensure PII data masking and encryption in your ETL pipeline?

Mask at ingestion: hash (`SHA-256`), tokenize, or redact PII fields. Encrypt storage with KMS. Restrict column access via Unity Catalog or IAM. Implement right-to-erasure via Delta DELETE. Log all access. Never store PII in logs.

12. You are asked to design a slowly changing dimension (SCD) Type 2 pipeline in Glue. How will you track history while keeping query performance acceptable?

Read new records ³ compare hash with existing. For changed records: set `end_date` = today, `is_current = False` on old rows. Insert new row with new values and `is_current = True`. Use MERGE in Delta or staging table + SQL in Redshift.

13. Your batch ETL pipeline has strict SLAs (must finish before 8 AM daily). What steps will you take to ensure timely completion even when data volume doubles?

Profile job bottlenecks. Pre-warm clusters. Increase DPUs/executors for peak days. Partition and compact input data. Add SLA monitoring (CloudWatch alarm, Airflow SLA). Add 30-min buffer. Use parallel processing for independent steps.

14. During a production release, your Spark job failed because of missing Python packages. How would you package and deploy external dependencies reliably in Glue/Lambda/Spark?

For Glue: use `--additional-python-modules` or upload `.whl` to S3. For Lambda: bundle in ZIP or Lambda Layer. For Spark: use `--packages` (Maven) or distribute via `spark.submit.pyFiles`. Store dependency versions in requirements.txt and pin them.

15. A downstream analytics team reports inconsistent results from the same dataset at different times of the day. How do you debug and ensure data consistency?

Check if upstream pipeline has multiple runs writing to same partition (overwrite race condition). Verify data is not modified mid-query. Use Delta snapshot isolation. Add row count and checksum comparison per pipeline run. Implement dataset versioning.

16. Your team wants to move from a monolithic ETL pipeline to modular micro-ETL jobs. What benefits will this provide, and how would you implement it?

Benefits: independent deployment, easier debugging, reusability, smaller blast radius on failure. Implement by splitting by domain/source. Each micro-ETL handles one source/transform. Orchestrate with Airflow. Share common libraries via Python packages.

17. Your data pipeline is built on AWS services. A region-wide outage occurs. How will you design for fault tolerance and high availability?

Multi-region S3 replication. Redshift cross-region snapshots. Route 53 failover routing. Lambda deployed in multiple regions. Use AWS Global Accelerator. Design for async processing 4 SQS persists events if processing region is down.

18. The business team requests near real-time alerts when fraud transactions are detected. How would you implement this on AWS?

Transaction ³ Kinesis Data Streams ³ Lambda (ML fraud detection model) ³ if fraud score high ³ SNS alert ³ email/SMS/Slack. Also write results to DynamoDB for real-time lookup. Use Kinesis Firehose for S3 archival.

19. A data scientist asks you to provide training data from multiple sources (structured + unstructured). How do you design a pipeline to ensure clean, joined, and high-quality data?

Ingest structured (DB ³ Spark ³ Delta). Ingest unstructured (files/API ³ S3 ³ NLP preprocessing). Join on common entity ID. Run data quality checks. Write to feature table. Apply consistent schema and timestamps for reproducibility.

20. Everyday we load data and the job is scheduled, but today it failed. Data will not be loaded. What do you do?

Immediately check logs for root cause. Fix the issue. Manually trigger the pipeline for the missed date. Validate data loaded correctly. Notify downstream teams of the delay. Document the incident. Add preventive monitoring.

21. You have TABLE1 and TABLE2. Give the output of Cross Join, Left Outer Join, Right Outer Join, Full Outer Join.

Cross Join: all combinations (N×M rows). **Left Outer**: all T1 rows + matched T2 (NULLs where no match). **Right Outer**: all T2 rows + matched T1 (NULLs where no match). **Full Outer**: all rows from both, NULLs where no match on either side.

22. If you have a pipeline that has completed 40% of the job but then fails, how do you ensure the rest 60% completes without re-executing the first 40%?

Design idempotent checkpoints 4 write completed segments to intermediate storage (S3 prefix or Delta). On restart, check which checkpoint exists and skip those. Use Airflow task-level reruns. Glue Job Bookmarks handle this for incremental sources.

23. You have 100 stores connected via API and want data from all stores every 10 minutes with exception handling. How do you design this?

Use async Python (`asyncio` + `aiohttp`) or ThreadPoolExecutor for parallel API calls. Handle errors per store independently. Retry on failure. Store results in S3 partitioned by store + timestamp. Schedule with Airflow every 10 min.

24. You are preserving large files without using Spark. How do you approach this?

Use Python generators + chunked reading (`pd.read_csv(chunksize=10000)`). Process chunk by chunk 4 transform and write to output. Use Dask for parallel processing. For very large files, consider splitting into smaller files first.

25. Keeping a KPI in mind, explain how you are extracting, cleaning and loading data.

Extract: pull from source (API, DB, S3) filtered to relevant KPI timeframe. **Clean**: remove nulls, duplicates, outliers; standardize formats. **Load**: write to data warehouse partitioned by KPI dimension (date, region). Validate KPI metric matches expected range.

26. How do you restrict duplicate records from passing downstream in your pipeline?

Add deduplication step before writing to target: `df.dropDuplicates(['primary_key'])`. In Redshift: use staging table + MERGE with ON conflict key. In Delta: use MERGE INTO with match on primary key. Log and alert on high duplicate rates.

27. Design a pipeline for ecommerce 4 what tables and columns do you need?

Tables: `fact_orders` (order_id, customer_id, product_id, date_id, qty, amount), `dim_customer` (customer_id, name, segment, region), `dim_product` (product_id, category, price), `dim_date`. KPIs: revenue, AOV, conversion rate, churn by segment.

28. It takes 36 hours in Elastic Search for Oracle data to be ingested and processed. How do you optimize this?

Parallelize Oracle extraction using partitioned JDBC reads (partition by ID range). Use Spark for transformation instead of single-threaded ETL. Batch-index into Elasticsearch using `elasticsearch-hadoop` connector. Use index aliases for zero-downtime reindexing. Target: <4 hours.

29. You have a new client with different data vendors and 15 key metrics that change weekly or monthly. How would you set up an ETL pipeline?

Build metadata-driven ETL: store KPI definitions in a config table (metric name, formula, source column, refresh frequency). Pipeline reads config and executes dynamically. Adding/changing a KPI = update config table, no code change.

30. What is a CROSS JOIN?

A CROSS JOIN returns the Cartesian product of two tables — every row from the left table is combined with every row from the right table. No join condition is needed. Result rows = rows in Table A × rows in Table B.

Example

```
SELECT a.color, b.size
FROM colors a
CROSS JOIN sizes b;
```

If `colors` has 3 rows (Red, Blue, Green) and `sizes` has 2 rows (S, M), the result is 6 rows: (Red, S), (Red, M), (Blue, S), (Blue, M), (Green, S), (Green, M).

When to Use

Use cases: Generate all combinations (e.g., product variants), create test datasets, build date/dimension grids. Avoid on large tables — can produce millions of rows quickly.